In [ ]:
```
EXPERIMENT 7 – HANDLING OUTLIERS & MISSING VALUES IN A DATASET
PIN NO:-241UDS0101

AIM:-
To preprocess a dataset by removing duplicate records, detecting and
handling outliers using the IQR method, and filling missing values using
Pandas and Scikit-learn imputers.
```

In [ ]:
```
## Step 1: Import the Required Libraries
Run the installation line only if any library is missing.
```

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import SimpleImputer
```

In [ ]:
```
## Step 2: Load and Analyze the Dataset
Keep the CSV file in the same folder as this notebook.
```

In [2]:
```
retail_staff_241UDS0101 =
pd.read_csv("Experiment_7_241UDS0101_Retail_Staff_Dataset.csv")
retail_staff_241UDS0101.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 49 entries, 0 to 48
Data columns (total 10 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   PIN               49 non-null     object
 1   Staff_ID          49 non-null     object
 2   Branch            48 non-null     object
 3   Role              48 non-null     object
 4   Gender            48 non-null     object
 5   Monthly_Sales     47 non-null     float64
 6   Hours_Per_Week    48 non-null     float64
 7   Customer_Rating   48 non-null     float64
 8   Years_Experience  48 non-null     float64
 9   Training_Hours    48 non-null     float64
dtypes: float64(5), object(5)
memory usage: 4.0+ KB
```

In [3]:
```
retail_staff_241UDS0101.head()
```

Out[3]: Columns 1–8 of 11

	PIN	Staff_ID	Branch	Role	Gender	Monthly_Sales	Hours_Per_Week
0	241UDS0101	STF001	North	Stock Associate	Female	59353.0	41.0
1	241UDS0101	STF002	West	Delivery	Female	39347.0	25.0
2	241UDS0101	STF003	East	Delivery	Male	NaN	42.0
3	241UDS0101	STF004	North	Delivery	Male	53105.0	25.0
4	241UDS0101	STF005	Central	Sales Assistant	Non-binary	60338.0	49.0

Columns 9–11 of 11 · first column repeated

	Customer_Rating	Years_Experience	Training_Hours
0	4.91	5.0	26.0
1	3.63	6.0	8.0
2	3.60	4.0	15.0
3	4.32	3.0	30.0
4	3.39	1.0	14.0

In [4]:
```
retail_staff_241UDS0101.shape
```

Out[4]:
```
(49, 10)
```

In [ ]:
```
## Step 3: Find and Remove Duplicates
First count duplicate rows, then remove them.
```

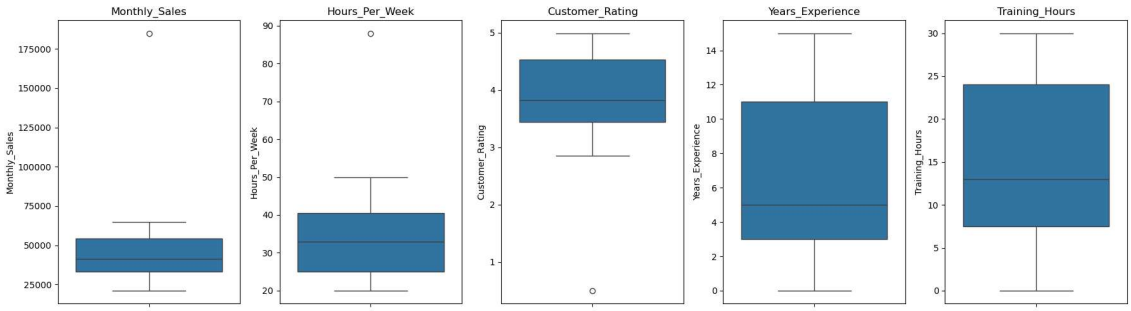
In [5]:
```
print("Duplicate rows:", retail_staff_241UDS0101.duplicated().sum())
retail_staff_241UDS0101 = retail_staff_241UDS0101.drop_duplicates()
print("Shape after removing duplicates:", retail_staff_241UDS0101.shape)
```

```
Duplicate rows: 1
Shape after removing duplicates: (48, 10)
```

In [ ]:
```
## Step 4: Identify Outliers Using Box Plots
Visualize the numeric attributes before handling outliers.
```

In [7]:
```
numeric_cols_241UDS0101 = [
    "Monthly_Sales", "Hours_Per_Week", "Customer_Rating",
    "Years_Experience", "Training_Hours"
]

fig, axes = plt.subplots(1, len(numeric_cols_241UDS0101), figsize=(18, 5))
for ax, col in zip(axes, numeric_cols_241UDS0101):
    sns.boxplot(y=retail_staff_241UDS0101[col], ax=ax)
    ax.set_title(col)
plt.tight_layout()
plt.show()
```



In [ ]:
```
## Step 5: Find Outliers Using the IQR Method
The IQR is Q3 - Q1. Values below Q1 - 1.5 × IQR or above Q3 + 1.5 × IQR are
flagged as outliers.
```

In [8]:
```
outlier_bounds_241UDS0101 = {}
for col in numeric_cols_241UDS0101:
    q1 = retail_staff_241UDS0101[col].quantile(0.25)
    q3 = retail_staff_241UDS0101[col].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    outlier_bounds_241UDS0101[col] = (lower, upper)
    count = ((retail_staff_241UDS0101[col] < lower) |
(retail_staff_241UDS0101[col] > upper)).sum()
    print(f"{col}: lower={lower:.2f}, upper={upper:.2f}, outliers={count}")
```

```
Monthly_Sales: lower=1780.12, upper=85779.12, outliers=1
Hours_Per_Week: lower=1.75, upper=63.75, outliers=1
Customer_Rating: lower=1.82, upper=6.16, outliers=1
Years_Experience: lower=-9.00, upper=23.00, outliers=0
Training_Hours: lower=-17.25, upper=48.75, outliers=0
```

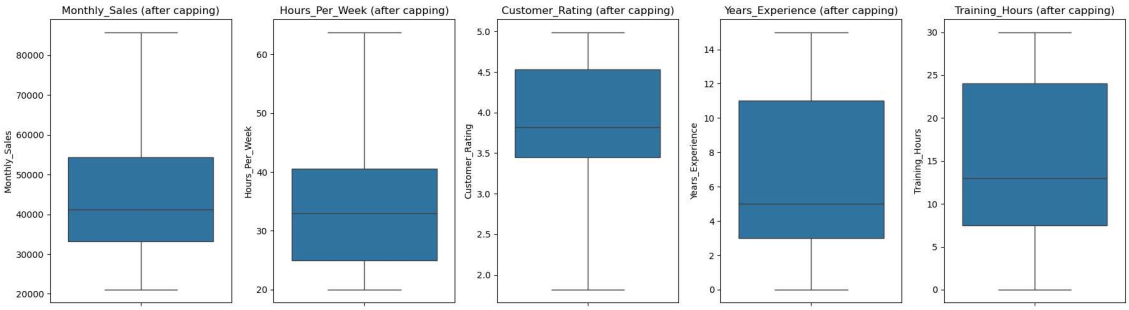
In [ ]:
```
## Step 6: Handle Outliers by Capping with IQR Bounds
To avoid dropping entire records, cap numeric values to the IQR lower and
upper bounds. Missing values remain missing for the imputation step.
```

In [11]:
```
for col in numeric_cols_241UDS0101:
    lower, upper = outlier_bounds_241UDS0101[col]
    retail_staff_241UDS0101[col] =
retail_staff_241UDS0101[col].clip(lower=lower, upper=upper)

print("Outlier capping completed.")
```

```
Outlier capping completed.
```

In [10]:
```
fig, axes = plt.subplots(1, len(numeric_cols_241UDS0101), figsize=(18, 5))
for ax, col in zip(axes, numeric_cols_241UDS0101):
    sns.boxplot(y=retail_staff_241UDS0101[col], ax=ax)
    ax.set_title(col + " (after capping)")
plt.tight_layout()
plt.show()
```



In [ ]:
```
## Step 7: Check Missing Values
Count missing values in every column.
```

In [12]:
```
print(retail_staff_241UDS0101.isnull().sum())
```

```
PIN                 0
Staff_ID            0
Branch              1
Role                1
Gender              1
Monthly_Sales       2
Hours_Per_Week      1
Customer_Rating     1
Years_Experience    1
Training_Hours      1
dtype: int64
```

In [ ]:
```
## Step 8: Fill Missing Numeric Values with the Mean
Use `SimpleImputer(strategy='mean')` for numeric columns.
```

In [13]:
```
numeric_imputer_241UDS0101 = SimpleImputer(strategy="mean")
retail_staff_241UDS0101[numeric_cols_241UDS0101] =
numeric_imputer_241UDS0101.fit_transform(
    retail_staff_241UDS0101[numeric_cols_241UDS0101]
)
print(retail_staff_241UDS0101[numeric_cols_241UDS0101].isnull().sum())
```

```
Monthly_Sales       0
Hours_Per_Week      0
Customer_Rating     0
Years_Experience    0
Training_Hours      0
dtype: int64
```

In [ ]:
```
## Step 9: Fill Missing Categorical Values with the Mode
Use the most frequent value in each categorical column.
```

In [14]:
```
categorical_cols_241UDS0101 = ["Branch", "Role", "Gender"]
categorical_imputer_241UDS0101 = SimpleImputer(strategy="most_frequent")
retail_staff_241UDS0101[categorical_cols_241UDS0101] =
categorical_imputer_241UDS0101.fit_transform(
    retail_staff_241UDS0101[categorical_cols_241UDS0101]
)
print(retail_staff_241UDS0101[categorical_cols_241UDS0101].isnull().sum())
```

```
Branch    0
Role      0
Gender    0
dtype: int64
```

In [ ]:
```
## Step 10: Verify the Cleaned Dataset
Check remaining missing values, dimensions, and preview the cleaned data.
```

In [15]:
```
print("Missing values after cleaning:")
print(retail_staff_241UDS0101.isnull().sum())
print("\nFinal shape:", retail_staff_241UDS0101.shape)
retail_staff_241UDS0101.head(10)
```

```
Missing values after cleaning:
PIN                 0
Staff_ID            0
Branch              0
Role                0
Gender              0
Monthly_Sales       0
Hours_Per_Week      0
Customer_Rating     0
Years_Experience    0
Training_Hours      0
dtype: int64

Final shape: (48, 10)
```

Out[15]: Columns 1–8 of 11

	PIN	Staff_ID	Branch	Role	Gender	Monthly_Sales	Hours_Per_Week
0	241UDS0101	STF001	North	Stock Associate	Female	59353.000000	41.0
1	241UDS0101	STF002	West	Delivery	Female	39347.000000	25.0
2	241UDS0101	STF003	East	Delivery	Male	43817.002717	42.0
3	241UDS0101	STF004	North	Delivery	Male	53105.000000	25.0
4	241UDS0101	STF005	Central	Sales Assistant	Non-binary	60338.000000	49.0
5	241UDS0101	STF006	East	Supervisor	Male	85779.125000	50.0
6	241UDS0101	STF007	North	Supervisor	Female	24523.000000	25.0
7	241UDS0101	STF008	South	Sales Assistant	Non-binary	61494.000000	38.0
8	241UDS0101	STF009	East	Sales Assistant	Non-binary	28381.000000	46.0
9	241UDS0101	STF010	North	Delivery	Male	58061.000000	38.0

Columns 9–11 of 11 · first column repeated

	Customer_Rating	Years_Experience	Training_Hours
0	4.910000	5.0	26.0
1	3.630000	6.0	8.0
2	3.600000	4.0	15.0
3	4.320000	3.0	30.0
4	3.390000	1.0	14.0
5	3.610000	11.0	3.0
6	3.480000	11.0	10.0
7	4.330000	15.0	20.0
8	3.895904	4.0	1.0
9	3.640000	9.0	16.0

In [ ]:
```
RESULT:-
The retail staff dataset for PIN 241UDS0101 was loaded and inspected.
Duplicate records were removed, numeric outliers were identified using the
IQR method and capped, and missing numeric and categorical values were
filled using mean and most-frequent imputers.
```